



LJ University
University with a Difference

DATA STRUCTURE USING JAVA

LECTURE NOTES FOR UNIT - 1

UNIT -1 INTRODUCTION TO DATA STRUCTURE

PREPARED BY:
MR. VISMAY SHAH

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Contents

1.1. Introduction	3
1.2. What is Data Structure?	4
1.3. Types of Data Structure	6
a. Primitive Data Structures	6
b. Non - Primitive Data Structures	6
c. Difference Between Primitive & Non-Primitive Data Structures	7
d. Linear Data Structure	8
e. Non-Linear Data Structure	8
f. Difference Between Linear and Non-Linear Data Structure	8
g. Classification of Data Structure Based on Memory Level	9
h. Classification of Data Structure Based on Data Stored	9
1.4. Why It is Important to Study Data Structures? [Need of Data Structure]	10
1.5. How Data Structure vary from Data Type?	12
1.6. Operation On Data Structures	13
1.7. Arrays	14
1.8. Characteristics of Array	15
1.9. Why is Array Data Structure needed?	16
1.10. Operations Performed on Array	17
1.11. Application of Array	18
1.12. How to Declare Array in JAVA	19
1.13. Size of Array	20
1.14. Address Calculation of One-Dimensional Array	21
1.15. Programs & Examples for Address Calculation in 1-D Array	23
1.16. Address Calculation of Two-Dimensional Array	26
1.17. Programs & Examples for Address Calculation in 2-D Array	29
1.18. Brief About Other Linear Data Structures	38
a. Stack	38
b. Queue	39
c. LinkedList	40
1.19. Non-linear data structure:	42
a. Trees	42

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

b. Graphs	43
1.20. Application of Data Structures	44



1.1. Introduction

To understand the concept of Data Structure let us consider a basic example –

For Example, As an engineering student You are using one single note book to prepare lecture notes for 3 subject (A, B & C). Now that there are 3 subjects against of 1 notebook – there are 2 methods to make notes.

Method 1 – You sequentially prepare the notes like

Page 1 to 5 – Subject A

Page 6 to 10 – Subject B

Page 11 to 20 – Subject C & Again

Page 21 to 25 – Subject A and so on.

Method 2 – You divide the entire notebook in to 3 section/parts

Page 1 to 50 – Subject A

Page 51 to 100 – Subject B

Page 101 to 150 – Subject C

Now, think of the scenario at the time of the exam! Which method will be helpful to you in preparing for the exam?

☐ **Method 2 is preferable.**

But why?

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

As you are aware that preparation for exam can be done by any of the method, but if we look at the aspect of time,

The Time taken to access the data is less in Method 2.

Now, Let us co-relate this to the technical part.

Apps and Sites like Amazon and Flipkart have huge amount of data. So, the point is How to Store/Save the data in such a manner that can be stored effectively, accessed in minimum time and storage of memory should be comparatively less.

THIS IS WHAT DATA STRUCTURE IS.

1.2. What is Data Structure?

- ◆ **A Data Structure is specific way of organizing data in a computer memory(cache, main, secondary) – so that memory can be used efficiently both in terms of time and space.**
- ◆ **A Data Structure is not only used for organizing data. It is also used for processing, retrieving and storing the data.**
- ◆ There are different basic and advanced types of data structures that are used in almost every program or software system that has been developed. So, we must have good knowledge of data structures.
- ◆ Various types of data structures have their own characteristics, features, applications, advantages, and disadvantages. So, you need to identify a data structure that is suitable for a particular task.

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Data structures mainly specifies the following 4 things

- 

01 **Organization of Data** How to organize or store the data in which structure?
For example Array, LinkedList etc.
- 

02 **Accessing Method** How you can reach to some elements for example – If
you store elements in array and you know the size of
each element and base address, you can reach up to any
element
- 

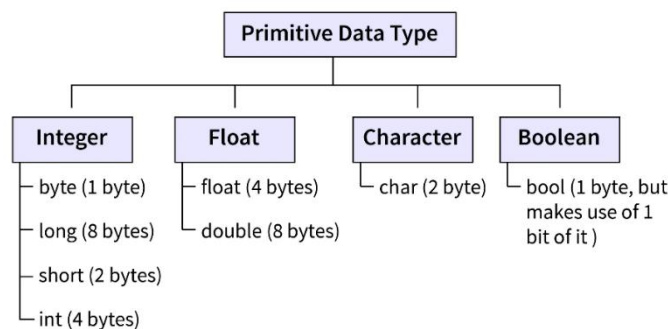
03 **Degree of Association** How many elements are dependent on How many other
elements.
- 

04 **Processing Method** Represents How to insert, update & delete any element
in Data Structure

1.3. Types of Data Structure

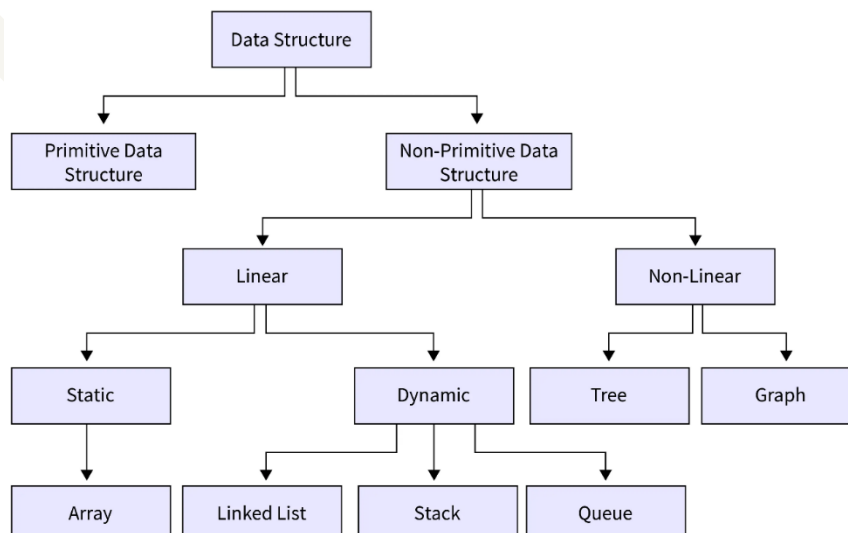
a. Primitive Data Structures

- ◆ Primitive data structures are the most basic data structures available in a programming language, such as integers, floating-point numbers, characters, and Booleans.
- ◆ These are operated at Machine Level.



b. Non - Primitive Data Structures

- ◆ Non-primitive data structures are complex data structures that are built using primitive data types.
- ◆ Non-primitive data structures are divided into linear and non-linear data structures. Arrays and linked lists are examples of linear data structure and trees, graph are examples of non-linear data structure.



c. Difference Between Primitive & Non-Primitive Data Structures

Primitive Data Structure	Non-Primitive Data Structure
Each primitive data structure can only hold one value of a specific type like integer, character, Boolean, etc.	The non-primitive data structure is a type of data structure that can store data of more than one type.
Examples of primitive data structures are integer, character, and float. s	Examples of non-primitive data structures are Array, linked list, and stack.
They don't have built-in methods to perform operations on the data.	Non-primitive data structures often come with built-in methods to add, remove, search, or manipulate the elements they hold.
Since Primitive Data Structure holds a single value, they occupy minimal memory space.	Their size can change at runtime depending on the data they store.

Think of primitive data structures like basic building blocks (like Lego bricks). They are simple and can only hold one value each. Non-primitive data structures are like more complex structures you can build with these bricks (like houses or cars). They allow you to combine different types of data and perform operations on them.

By understanding these differences, you can choose the right data structure for your programming needs. Primitive data structures are efficient for storing basic data, while non-primitive data structures are essential for organizing and manipulating complex data sets.

- ◆ The choice of data structure for a particular task depends on the type and amount of data to be processed, the operations that need to be performed on the data, and the efficiency requirements of the program.
- ◆ Efficient use of data structures can greatly improve the performance of a program, making it faster and more memory-efficient.
- ◆ A data structure is a particular way of organizing data in a computer so that it can be used effectively. The idea is to reduce the space and time complexities of different tasks.

d. Linear Data Structure

- ◆ Data Structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous & next adjacent elements, is called a Linear Data Structure.
- ◆ Examples of Linear Data Structure are Array, Stack, Queue, Linked list, etc.

e. Non-Linear Data Structure

- ◆ Data Structures where data elements are not placed sequentially or linearly are called Non-Linear Data Structure. In a Non-Linear Data Structure, we can't traverse all the elements in a single run only.
- ◆ Examples of Non-Linear Data Structures are Trees and Graph.

f. Difference Between Linear and Non-Linear Data Structure

Linear Data Structure	Non-Linear Data Structure
In a linear data structure, data elements are arranged in a linear order where each and every element are attached to its previous and next adjacent.	In a non-linear data structure, data elements are attached to many other elements. (Attached in hierarchically manner)
Its implementation is easy in comparison to non-linear data structure.	While its implementation is complex in comparison to linear data structure.
In linear data structure, data elements can be traversed in a single run only.	While in non-linear data structure, data elements can't be traversed in a single run only.
In a linear data structure, memory is not utilized in an efficient way.	While in a non-linear data structure, memory is utilized in an efficient way.
Its examples are: array, stack, queue, linked list, etc.	Its examples are: Trees, Graphs, Hash Tables, etc.

g. Classification of Data Structure Based on Memory Level

i. Static Data Structure

- ◆ Static Data Structures are those whose size of structures associated memory locations are fixed, at compile time.
- ◆ An example of this data structure is an Array. (Once size of Array is given, it cannot be changed.)

ii. Dynamic Data Structure

- ◆ In dynamic data structure, the size is not fixed. It can be randomly updated (increased or decreased) during the runtime which may be considered efficient concerning the memory (space) complexity of the code.
- ◆ Examples of this data structure are Queue, Stack, etc.

h. Classification of Data Structure Based on Data Stored

i. Homogeneous

- ◆ In Homogeneous Data Structure – All the elements are of same type. For example - Array

ii. Non-Homogeneous

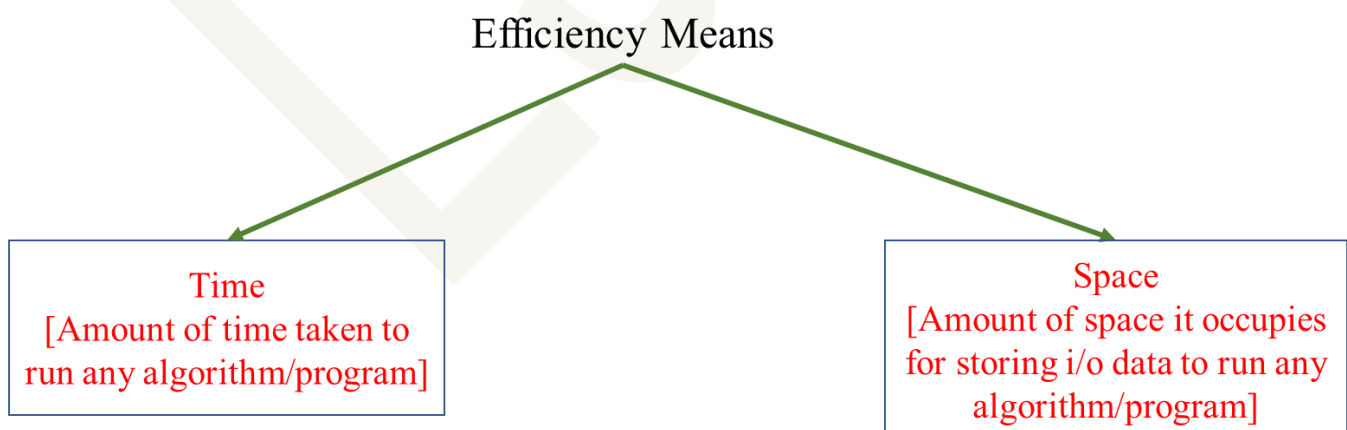
- ◆ In Non - Homogeneous Data Structure – The elements may or may not be of the same types. For example – Classes / Objects

1.4. Why It is Important to Study Data Structures? [Need of Data Structure]

- ◆ As applications are becoming more complex and the amount of data is increasing day by day, which may cause problems with processing speed, searching data, handling multiple requests etc.
 - ◆ Data structure provides a way of organizing, managing, and storing data efficiently. With the help of data structure, the data items can be traversed easily.
 - ◆ Data structure provides efficiency, reusability and abstraction. It plays an important role in enhancing the performance of a program because the main function of the program is to store and retrieve the user's data as fast as possible.
-
- ◆ Computer Science deals with solving a problem correctly in the form of Algorithm which then can be converted into a program, in most efficient time and memory.



- ◆ Objective of Data Structure is to teach you “How to Code Efficiently.”



DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

In Summary, we need to learn and understand Data Structures because

- ◆ It save storage memory space.
- ◆ Data structure's modification is easy.
- ◆ It requires less time.
- ◆ Data representation is easy.
- ◆ It becomes easy access to the large database.

1.5. How Data Structure vary from Data Type?

We already have learned about data structure. Many times, what happens is that people get confused between data type and data structure. So, let's see a few differences between data type and data structure to make it clear.

Data Type	Data Structure
The data type is the form of a variable to which a value can be assigned. It defines that the particular variable will assign the values of the given data type only.	The non-primitive data structure is a type of data structure that can store data of more than one type.
It can hold value but not data. Therefore, it is dataless.	It can hold multiple types of data within a single object.
There is no time complexity in the case of data types.	In data structure objects, time complexity plays an important role.
In the case of data types, the value of data is not stored because it only represents the type of data that can be stored.	While in the case of data structures, the data and its value acquire the space in the computer's main memory. Also, a data structure can hold different kinds and types of data within one single object.
Data type examples are int, float, double, etc.	Data structure examples are stack, queue, tree, etc.

1.6. Operation On Data Structures

The major operations performed on data structures are:

Insertion - Insertion means adding new details or new node into the data structure.

Deletion - Deletion means removing a node from the data structure.

Traversal - Traversing means accessing each node exactly once so that the nodes of a data structure can be processed. Traversing is also called as visiting.

Searching - Searching means finding the location of node for a given key value.

Apart from the four operations mentioned above, there are two more operations occasionally performed on data structures. They are:

Sorting - Sorting means arranging the data in a particular order.

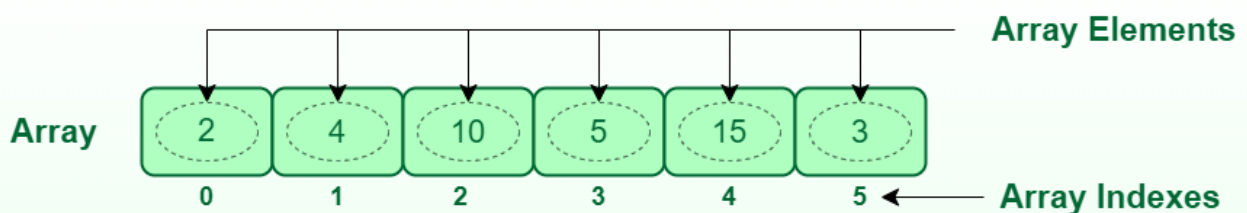
Merging - Merging means joining two lists.

1.7. Arrays

- ◆ An array is a collection of items of the same variable type stored that are stored at **contiguous memory locations**.
- ◆ The idea is to store multiple items of the same type together. This makes it easier to calculate the position of each element by simply adding an offset (**size of individual element**) to a base value.
- ◆ It's one of the most popular and simple data structures and is often used to implement other data structures.
- ◆ Each item in an array is indexed starting with 0. There are different operations possible in an array, like Searching, Sorting, Inserting, Traversing, Reversing, and Deleting.

Basic terminologies of array -

- ◆ **Array Index:** In an array, elements are identified by their indexes. Array index starts from 0.
- ◆ **Array Element:** Elements are items stored in an array and can be accessed by their index.
- ◆ **Array Length:** The length of an array is determined by the number of elements it can contain.



Memory Location									
200	201	202	203	204	205	206	▪	▪	▪
U	B	F	D	A	E	C	▪	▪	▪
0	1	2	3	4	5	6	▪	▪	▪
Index									

Note – Memory Location Number 200, 201 and so on are just provided for the example purpose only.

Array is a simplest data structure that makes use of computed address to locate its elements.

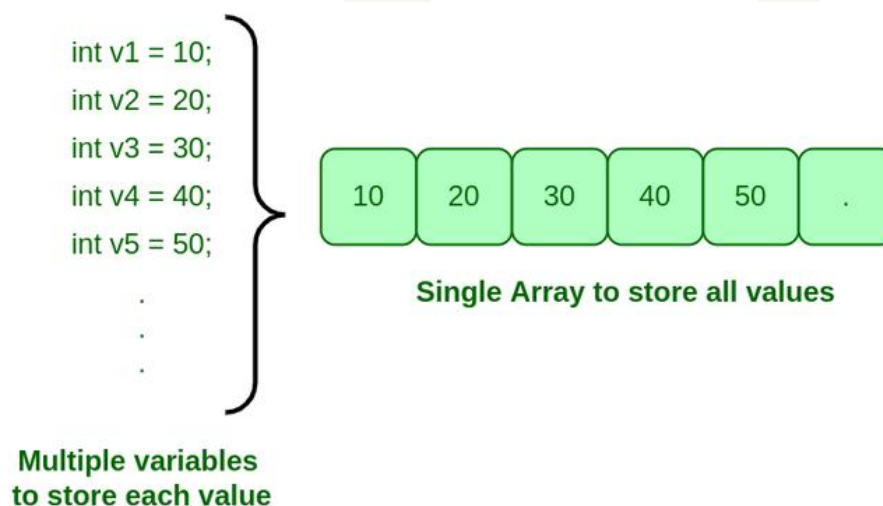
An array size is fixed and therefore requires a fixed number of memory locations.

1.8. Characteristics of Array

- ◆ Arrays use an index-based data structure which helps to identify each of the elements in an array easily using the index.
- ◆ If a user wants to store multiple values of the same data type, then the array can be utilized efficiently.
- ◆ An array can also handle complex data structures by storing data in a two-dimensional array.
- ◆ An array is also used to implement other data structures like Stacks, Queues, Heaps, Hash tables, etc.
- ◆ The search process in an array can be done very easily.

1.9. Why is Array Data Structure needed?

- ◆ Assume there is a class of five students and if we have to keep records of their marks in examination then, we can do this by declaring five variables individual and keeping track of records but what if the number of students becomes very large, **it would be challenging to manipulate and maintain the data.**
- ◆ What it means is that, we can use normal variables (v1, v2, v3, ..) when we have a small number of objects. But if we want to store a large number of instances, it becomes difficult to manage them with normal variables. The idea of an array is to represent many instances in one variable..



1.10. Operations Performed on Array

Initialization: An array can be initialized with values at the time of declaration or later using an assignment statement.

Accessing elements: Elements in an array can be accessed by their index, which starts from 0 and goes up to the size of the array minus one.

Searching for elements: Arrays can be searched for a specific element using linear search or binary search algorithms.

Sorting elements: Elements in an array can be sorted in ascending or descending order using algorithms like bubble sort, insertion sort, or quick sort.

Inserting elements: Elements can be inserted into an array at a specific location, but this operation can be time-consuming because it requires shifting existing elements in the array.

Deleting elements: Elements can be deleted from an array by shifting the elements that come after it to fill the gap.

Updating elements: Elements in an array can be updated or modified by assigning a new value to a specific index.

Traversing elements: The elements in an array can be traversed in order, visiting each element once.

These are some of the most common operations performed on arrays. The specific operations and algorithms used may vary based on the requirements of the problem and the programming language used.

1.11. Application of Array

- ◆ An array is used in solving matrix problems.
- ◆ Database records are also implemented by an array.
- ◆ It helps in implementing a sorting algorithm.
- ◆ It is also used to implement other data structures like Stacks, Queues, Heaps, Hash tables, etc.
- ◆ An array can be used for CPU scheduling.
- ◆ It can be applied as a lookup table in computers.
- ◆ Arrays can be used in speech processing where every speech signal is an array.
- ◆ The screen of the computer is also displayed by an array. Here we use a multidimensional array.
- ◆ The array is used in many management systems like a library, students, parliament, etc.
- ◆ The array is used in the online ticket booking system. Contacts on a cell phone are displayed by this array.
- ◆ In games like online chess, where the player can store his past moves as well as current moves. It indicates a hint of position.
- ◆ To save images in a specific dimension in the android like 360*1200.

Real-Life Application of Array

- ◆ An array is frequently used to store data for mathematical computations.
- ◆ It is used in image processing.
- ◆ It is also used in record management.
- ◆ Book pages are also real-life examples of an array.
- ◆ It is used in ordering boxes as well.

1.12. How to Declare Array in JAVA

```
datatype ArrayName[]=new datatype[size];
```

OR

```
int data[10] = {1,2,3,4,5,6,7,8,9,10};
```

Note – Students can use any other valid format as studied in JAVA-I.

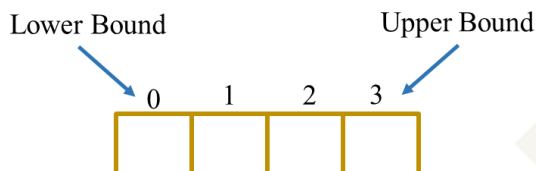
1.13. Size of Array

$$\text{Number of Elements} = \text{Upper Bound} - \text{Lower Bound} + 1$$

Upper Bound = Index of Last Element of Array

Lower Bound = Index of First Element of Array

Default Case

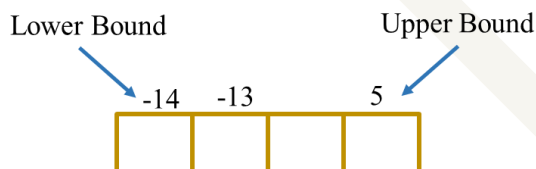


$$\text{Number of Elements} = \text{Upper Bound} - \text{Lower Bound} + 1$$

$$\text{Number of Elements} = 3 - 0 + 1 = 4$$

Hence Total Elements = 4.

Take another Array A[-14....5]



$$\text{Number of Elements} = \text{Upper Bound} - \text{Lower Bound} + 1$$

$$\text{Number of Elements} = 5 - (-14) + 1$$

Hence Total Elements = 20.

1.14. Address Calculation of One-Dimensional Array

- ◆ To Understand the concept of Address Calculation – let us take an example of array ARR [0 to 9]. Now, When you initialize and declare the Array in Java – it will have some address in the memory.
- ◆ Here in our case, we are considering that the Array ARR is Integer Array hence each element of it will occupy 4 bytes of memory. Also, we have assumed that the Address of the 1st element which is ARR[0] (Known as Base Address) is 1100.
- ◆ **Note – In Java memory addresses are in form of HASH CODES which are not known to us while in C we can know the memory address of a stored elements in array.**

Hence the Address can be calculated and represented as following:

Address in the Memory	1100	1104	1108	1112	1116	1120	1124	1128	1132	1136
Elements / Values	10	20	30	40	50	60	70	80	90	100
Array Index	0	1	2	3	4	5	6	7	8	9

- ◆ In our example, we can calculate the Address of any element by adding 4 bytes, but what if our array was of ARR [0 to 499] and we required the address of ARR[342] – In such scenario direct calculation will not work.

In such cases we need to use the following formula:

$$\text{LOC}(\text{ARR}[i]) = \text{Base Address of ARR} + (i) * \text{Size of Single Element}$$

- Where **i** is the element whose address we need to find.
- So, using this formula we can find ARR[342] as following:
- Based Address = 1100
- Size of Single Element = 4 bytes
- $\text{LOC}(\text{ARR}[i]) = 1100 + (342) * 4 = 2468$

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

◆ Now, Above formula only works when the Array Index starts from 0. But What if given Array ARR is **[-5 to 345]** & we need to find the address of **ARR[100]** ?

In such cases The Formula will slightly modified as

$$\text{LOC}(\text{ARR}[i]) = \text{Base Address of ARR} + (i - \text{LB}) * \text{Size of Single Element}$$

- Where **LB** stands for **Lower Limit or Lower Bound of subscript**.
- So, Here in our case **LB will be -5**. Now, Applying the formula
- Considering Based Address = 1100
- Size of Single Element = 4 bytes
- Lower Bound Subscript = -5
- $\text{LOC}(\text{ARR}[i]) = 1100 + (100 - (-5)) * 4 = 1520$

1.15. Programs & Examples for Address Calculation in 1-D Array

Q.1 - Write a Program to find the base address of the i^{th} element the user provides for a given array.

Note – Take the size of the Array, Base Address, and Single element Size from the user. If the user Enters a number greater than the Total size of Array – An Error Message should be displayed in the output.

Solution:

```

1  import java.util.*;
2
3  class run1 {
4      public static void main(String Args[]) {
5          Scanner sc = new Scanner(System.in); // Scanner class to take inputs from user
6          System.out.println("Enter The Total size of Array:");
7          int n = sc.nextInt(); // this will provide the total size of the array
8          System.out.println("What value shall be considered as base value?");
9          int baseadd = sc.nextInt(); // this will provide what to be considered as base
          address
10         System.out.println("Enter the size of Single Element?");
11         int size = sc.nextInt(); // this will provide the size of a single element
12         System.out.println("Enter the Position of the Element for which you want to find
          the Address:");
13         int i = sc.nextInt(); // this will provide the i-th element of which we want to
          find the address
14         System.out.println("Enter the LowerBound Index Value:");
15         int Lowerbound = sc.nextInt(); // this will provide the LowerBound Index Value
16
17         // Checking valid index range
18         if (i >= Lowerbound && i < Lowerbound + n) {
19             int ans = (baseadd + (i - Lowerbound) * size);
20             System.out.println("Address of your given element is " + ans);
21         } else {
22             System.out.println("Invalid Argument: Position is out of the valid array
          index range.");
23         }
24     }
25 }

```

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

So, we can say that the general formula to Locate the i^{th} element of the Array A will be

$$\text{LOC}(A[i]) = \text{Base Address of A} + (i - \text{LB}) * \text{Size of Single Element}$$

Note - In the above program we have calculated the Address of the i^{th} element with reference to 1st element (element who had index number 0) but what if the Array is given from Index number 1300 to 1900 just as in the next example?

Q.2: Given the base address of an array A[1300.....1900] as 1020 and the size of each element is 2 bytes in the memory, find the address of A[1700]?

Solution:

Given that

- Base Address (B) = 1020
- Storage size of one element store in any array (W) = 2 bytes
- Subset of element whose address to be found (i^{th} Element) = 1700
- Take note that Starting point was given at 1300. So, we shall consider it Lower Limit or Lower Bound of subscript (LB) = 1300.

So, the formula will be slightly changed to,

$$\text{LOC}(A[i]) = \text{Base Address of A} + (i - \text{LB}) * \text{Size of Single Element}$$

$$\begin{aligned}\text{Address of A [1700]} &= 1020 + 2 * (1700 - 1300) \\ &= 1020 + 2 * (400) \\ &= 1020 + 800\end{aligned}$$

$$\text{Address of A [1700]} = 1820$$

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE



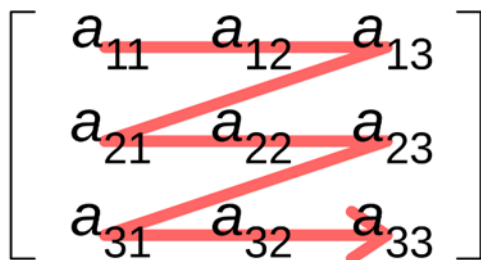
1.16. Address Calculation of Two-Dimensional Array

The elements of a Two-dimensional array can be arranged in two ways.

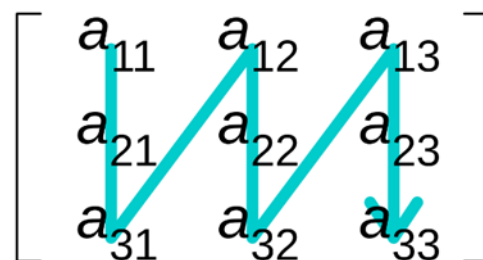
1. Row Major Representation
2. Column Major Representation

Let's make it simple with the below images.

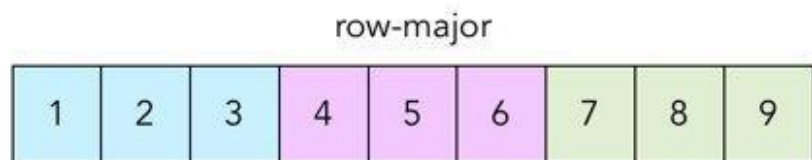
Row-major order



Column-major order



1	2	3
4	5	6
7	8	9



1	2	3
4	5	6
7	8	9



DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Row major Representation:

Row major ordering assigns successive elements, moving across the rows and then down the next row, to successive memory locations. In simple language, the elements of an array are stored in a Row-Wise fashion.

If we want to store element 10,20,30,40,50,60,70,80 and if size of array is a[3][4] then,

	0	1	2	3
0	10	20	30	40
1	50	60	70	80
2				

To find the address of the element using row-major order uses the following formula:

$$\text{Address of } A[i][j] = B + W * ((i - LR) * N + (j - LC))$$

Where,

i = Row Subset of an element whose address to be found,

j = Column Subset of an element whose address to be found,

B = Base address,

W = Storage size of one element store in an array (in byte),

LR = Lower Limit of row/start row index of the matrix (If not given assume it as zero),

LC = Lower Limit of column/start column index of the matrix (If not given assume it as zero),

N = Number of columns given in the matrix.

What does the formula says

- (i-LR) = How many rows are there before ith element
- (j-LC) = How many columns are there before jth element
- [(i-LR)*N + (j-LC)] = How many element are there before i and j

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Column major Representation:

If elements of an array are stored in a column-major fashion means moving across the column and then to the next column then it's in column-major order..

If we want to store element 10,20,30,40,50,60,70,80 and if size of array is a[3][4] then,

	0	1	2	3
0	10	40	70	
1	20	50	80	
2	30	60		

To find the address of the element using column-major order use the following formula:

$$\text{Address of } A[i][j] = B + W * ((j - LC) * M + (i - LR))$$

Where,

i = Row Subset of an element whose address to be found,

j = Column Subset of an element whose address to be found,

B = Base address,

W = Storage size of one element store in an array (in byte),

LR = Lower Limit of row/start row index of matrix(If not given assume it as zero),

LC = Lower Limit of column/start column index of matrix(If not given assume it as zero),

M = Number of rows given in the matrix.

What does the formula says

- (i-LR) = How many rows are there before ith element
- (j-LC) = How many columns are there before jth element
- [(i-LR + (j-LC)*M)] = How many element are there before i and j

1.17. Programs & Examples for Address Calculation in 2-D Array

Q.3 - Consider an integer array int a[3][4].if the base address is 1050,find the address of the element a[2][2] with row major and column major representation of array.

Solution:

1. Row Major System:

$$\text{Address of } A[i][j] = B + W * ((i - LR) * N + (j - LC))$$

Where,

i = Row Subset of an element whose address to be found = 2

j = Column Subset of an element whose address to be found = 2

B = Base address = 1050

W = Storage size of one element store in an array (in byte) = 4 (Not Given Assumed 4 of Int)

LR = Lower Limit of row/start row index of the matrix = 0 (Not Given)

LC = Lower Limit of column/start column index of the matrix = 0 (Not Given)

N = Number of columns given in the matrix = 4

Putting All the values in Formula

$$\text{Address of } A[i][j] = B + W * ((i - LR) * N + (j - LC))$$

$$A[2][2] = 1050 + 4 * ((2 - 0) * 4 + (2 - 0))$$

$$A[2][2] = 1090$$

	0	1	2	3
0	1050	1054	1058	1062
1	1066	1070	1074	1078
2	1082	1086	1090	

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

2. Column Major System:

$$\text{Address of } A[i][j] = B + W * ((j - LC) * M + (i - LR))$$

Where,

i = Row Subset of an element whose address to be found = 2

j = Column Subset of an element whose address to be found = 2

B = Base address = 1050

W = Storage size of one element store in an array (in byte) = 2 (Not Given Assumed 2 of Int)

LR = Lower Limit of row/start row index of the matrix = 0 (Not Given)

LC = Lower Limit of column/start column index of the matrix = 0 (Not Given)

M = Number of rows given in the matrix = 3

Putting All the values in Formula

$$\text{Address of } A[i][j] = B + W * ((j - LC) * M + (i - LR))$$

$$\text{Address of } A[2][2] = 1050 + 2 * ((2 - 0) * 3 + (2 - 0))$$

$$\text{Address of } A[2][2] = 1066$$

	0	1	2	3
0	1050	1056	1062	
1	1052	1058	1064	
2	1054	1060	1066	

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Q.4 - Write a Program for the following data - Consider an integer array `int a[3][4]`.if the base address is 1050,find the address of the element `a[2][2]` with row major and column major representation of array. Consider the size of each element 2 bytes. Calculate only by row-major method.

Solution:

```
1  import java.util.*;
2  class run
3  {
4      public static void main(String Args[])
5      {
6          Scanner sc= new Scanner(System.in); //Scanner class to take inputs from user//
7          System.out.println("Enter The Total Number of Rows for the Array:");
8          int r = sc.nextInt();
9          System.out.println("Enter The Total Number of Columns for the Array:");
10         int c = sc.nextInt();
11         System.out.println("What value shall be considered as base value?");
12         int baseadd=sc.nextInt();
13         System.out.println("Enter the size of Single Element?");
14         int size=sc.nextInt();
15         System.out.println("Enter the row position for the Element for which you want to find the Address:");
16         int i=sc.nextInt();
17         System.out.println("Enter the Column position for the Element for which you want to find the Address:");
18         int j=sc.nextInt();
19         System.out.println("Enter the starting Row Index of the Matrix:");
20         int LR=sc.nextInt();
21         System.out.println("Enter the starting Column Index of the Matrix:");
22         int LC=sc.nextInt();
23         if(i<=r && j<=c) //Checking the condition that element lies within the Array//
24         {
25             int ans = (baseadd+size*((i-LR)*c + (j-LC)));
26             System.out.println("Address of your given element is "+ans);
27         }
28         else //if not then error message shall be printed//
29         {
30             System.out.println("You have Entered Value of i & j incorrect");
31         }
32     }
33 }
```

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Q.5 - Write a Program for the following data - Consider an integer array `int a[3][4]`.if the base address is 1050,find the address of the element `a[2][2]` with row major and column major representation of array. Consider the size of each element 2 bytes. Calculate only by column-major method.

Solution:

```
1  import java.util.*;
2  class run
3  {
4      public static void main(String Args[])
5      {
6          Scanner sc= new Scanner(System.in); //Scanner class to take inputs from user//
7          System.out.println("Enter The Total Number of Rows for the Array:");
8          int r = sc.nextInt();
9          System.out.println("Enter The Total Number of Columns for the Array:");
10         int c = sc.nextInt();
11         System.out.println("What value shall be considered as base value?");
12         int baseadd=sc.nextInt();
13         System.out.println("Enter the size of Single Element?");
14         int size=sc.nextInt();
15         System.out.println("Enter the row position for the Element for which you want to find the Address:");
16         int i=sc.nextInt();
17         System.out.println("Enter the Column position for the Element for which you want to find the Address:");
18         int j=sc.nextInt();
19         System.out.println("Enter the start Row Index of the Matrix:");
20         int LR=sc.nextInt();
21         System.out.println("Enter the start Column Index of the Matrix:");
22         int LC=sc.nextInt();
23         if(i<=r && j<=c) //Checking the condition that element lies within the Array//
24         {
25             int ans = (baseadd+size*((i-LR) + (j-LC)*r));
26             System.out.println("Address of your given element is "+ans);
27         }
28         else //if not then error message shall be printed//
29         {
30             System.out.println("You have Entered Value of i & j incorrect");
31         }
32     }
33 }
```

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Q.6- Write a Program for the following data - Consider an integer array `int a[3][4]`.if the base address is 1050,find the address of the element `a[2][2]` with row major and column major representation of array. Consider the size of each element 2 bytes. Get Input from user to perform calculation by Row-Major or Column-Major.

```
1  import java.util.*;
2  class run
3  {
4      public static void main(String Args[])
5      {
6          Scanner sc= new Scanner(System.in);
7          System.out.println("Enter The Total Number of Rows for the Array:");
8          int r = sc.nextInt();
9          System.out.println("Enter The Total Number of Columns for the Array:");
10         int c = sc.nextInt();
11         System.out.println("What value shall be considered as base value?");
12         int baseadd=sc.nextInt();
13         System.out.println("Enter the size of Single Element?");
14         int size=sc.nextInt();
15         System.out.println("Enter the row position for the Element for which you want to find the Address:");
16         int i=sc.nextInt();
17         System.out.println("Enter the Column position for the Element for which you want to find the Address:");
18         int j=sc.nextInt();
19         System.out.println("Enter the start Row Index of the Matrix:");
20         int LR=sc.nextInt();
21         System.out.println("Enter the start Column Index of the Matrix:");
22         int LC=sc.nextInt();
23         System.out.println("Press 1 to Calculate by Row-Major Method");
24         System.out.println("Press 2 to Calculate by Column-Major Method");
25         int method=sc.nextInt();
26         if(method == 1)
27         {
28             if(i<=r && j<=c)
29             {
30                 int ans = (baseadd+size*((i-LR)*c + (j-LC)));
31                 System.out.println("Address of your given element is "+ans);
32             }
33             else
34             {
35                 System.out.println("You have Entered Value of i & j incorrect");
36             }
37         }
38         if(method == 2)
39         {
40             if(i<=r && j<=c)
41             {
42                 int ans = (baseadd+size*((i-LR) + (j-LC)*r));
43                 System.out.println("Address of your given element is "+ans);
44             }
45             else
46             {
47                 System.out.println("You have Entered Value of i & j incorrect");
48             }
49         }
50     }
51 }
```

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Q.7- A 2-D array defined as $A[r, c]$ where $1 \leq r \leq 4$, $5 \leq c \leq 8$, requires 2 bytes of memory for each element. If the array is stored in Row-major order form, calculate the address of $A[3,7]$ given the Base address as 2000.

Row Major System:

$$\text{Address of } A[i][j] = B + W * ((i - LR) * N + (j - LC))$$

Where,

i = Row Subset of an element whose address to be found = 3

j = Column Subset of an element whose address to be found = 7

B = Base address = 2000

W = Storage size of one element store in an array (in byte) = 2

LR = Lower Limit of row/start row index of the matrix = 1

LC = Lower Limit of column/start column index of the matrix = 5

N = Number of columns given in the matrix = 4

Putting All the values in Formula

$$\text{Address of } A[i][j] = B + W * ((i - LR) * N + (j - LC))$$

$$A[3][7] = 2000 + 2 * ((3 - 1) * 4 + (7 - 5))$$

$$A[3][7] = 2020$$

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Q.8: Given a two dimensional array $Z1(2:9, 9:18)$ stored in column major order with base address 100 and size of each element is 4 bytes, find address of The element $Z1(4, 12)$.

Column Major System:

$$\text{Address of } A[i][j] = B + W * ((j - LC) * M + (i - LR))$$

Where,

i = Row Subset of an element whose address to be found = 4

j = Column Subset of an element whose address to be found = 12

B = Base address = 100

W = Storage size of one element store in an array (in byte) = 4

LR = Lower Limit of row/start row index of the matrix = 2

LC = Lower Limit of column/start column index of the matrix = 9

M = Number of rows given in the matrix = 8

Putting All the values in Formula

$$\text{Address of } A[i][j] = B + W * ((j - LC) * M + (i - LR))$$

$$\text{Address of } A[4][12] = 100 + 4 * ((12 - 9) * 8 + (4 - 2))$$

$$\text{Address of } A[4][12] = 204$$

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Q.9 - Given an array, arr[1.....10][1.....15] with base value 100 and the size of each element is 1 Byte in memory. Find the address of arr [8][6] with the help of row-major order & column-major order?

Row Major System:

$$\text{Address of } A[i][j] = B + W * ((i - LR) * N + (j - LC))$$

Where,

i = Row Subset of an element whose address to be found = 8

j = Column Subset of an element whose address to be found = 6

B = Base address = 100

W = Storage size of one element store in an array (in byte) = 1

LR = Lower Limit of row/start row index of the matrix = 1

LC = Lower Limit of column/start column index of the matrix = 1

N = Number of columns given in the matrix = 15

Putting All the values in Formula

$$\text{Address of } A[i][j] = B + W * ((i - LR) * N + (j - LC))$$

$$A[8][6] = 100 + 1 * ((8 - 1) * 15 + (6 - 1))$$

$$A[8][6] = 210$$

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Column Major System:

$$\text{Address of } A[i][j] = B + W * ((j - LC) * M + (i - LR))$$

Where,

i = Row Subset of an element whose address to be found = 8

j = Column Subset of an element whose address to be found = 6

B = Base address = 100

W = Storage size of one element store in an array (in byte) = 1

LR = Lower Limit of row/start row index of the matrix = 1

LC = Lower Limit of column/start column index of the matrix = 1

M = Number of rows given in the matrix = 15

Putting All the values in Formula

$$\text{Address of } A[i][j] = B + W * ((j - LC) * M + (i - LR))$$

$$\text{Address of } A[8][6] = 100 + 1 * ((6 - 1) * 10 + (8 - 1))$$

$$\text{Address of } A[8][6] = 157$$

1.18. Brief About Other Linear Data Structures

a. Stack

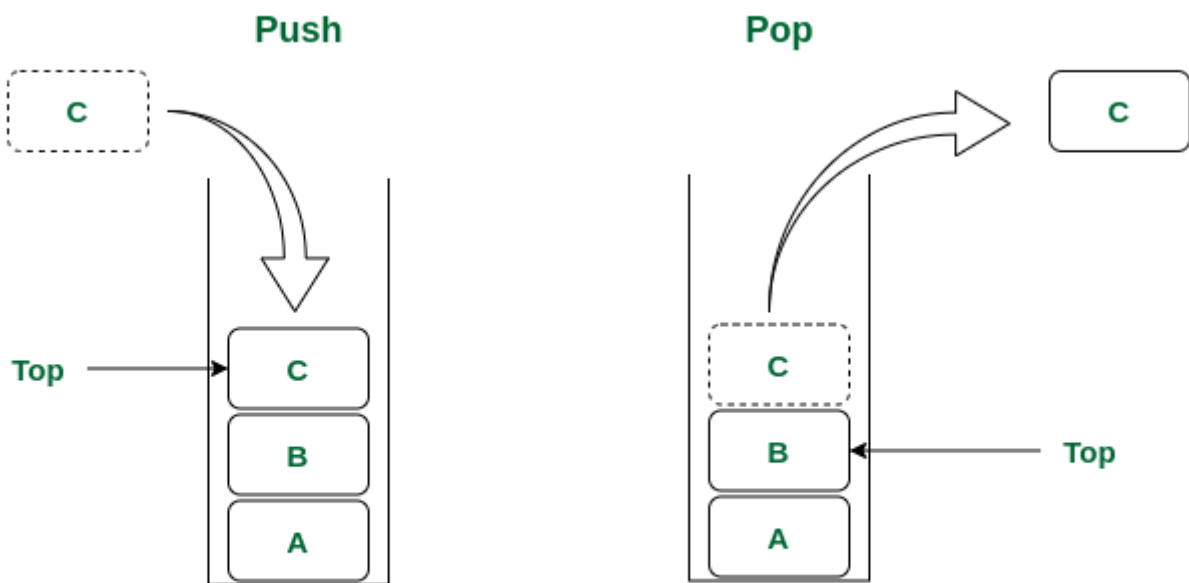
A stack is a linear list of data elements in which insertion of new element and deletion of the element occurs at the one end. This end is called as **TOP** of the STACK.

Stack is a linear data structure that follows a particular order in which the operations are performed.

The order may be **LIFO(Last In First Out)** or **FILO(First In Last Out)**. LIFO implies that the element that is inserted last, comes out first and FILO implies that the element that is inserted first, comes out last.

The operation of inserting the new element in the stack and the deletion of the element from the stack is called as **PUSH & POP** Respectively.

It is a linear list in which insertion and deletion take place only at one end. It is used to evaluate different expressions.



Stack Data Structure

There are many real-life examples of a stack. Consider an example of plates stacked over one another in the canteen. The plate which is at the top is the first one to be removed, i.e. the plate which has been placed at the bottommost position remains in the stack for the longest period of time. So, it can be simply seen to follow LIFO(Last In First Out)/FILO(First In Last Out) order.

b. Queue

A Queue is defined as a linear data structure that is open at both ends and the operations are performed in **First In First Out (FIFO)** order.

Queue is a linear data structure in which the insertion of the new element occurs at the one end called as – **REAR** and the deletion of the element from the queue occurs at another end called as **FRONT**.

It is a linear list in which insertion takes place at once end and deletion takes place at other end.

It is generally used to schedule a job in operating systems and networks.



Queue Data Structure

A Queue is like a line waiting to purchase tickets, where the first person in line is the first person served. (i.e. First come first serve).

c. LinkedList

Linked List is a collection of a similar data elements, called nodes with each node containing some data and pointer to pointing the other nodes in list.

Nodes of the linked list are not containing continuous memory location, instead of that they are stored anywhere in the memory. A list is an ordered list, which consists of different data items connected by means of a link or pointer.

This type of list is also called a linked list. A linked list may be a single list or double linked list.

i. Singly Linked List:

A singly LinkedList is used to traverse among the nodes in one direction. (The pointer field of the last node contains Null.

ii. Doubly Linked List:

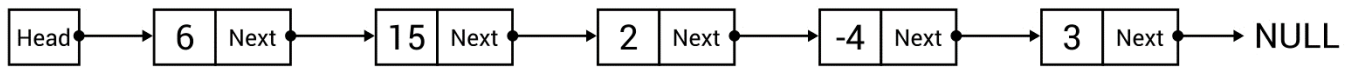
A doubly LinkedList is used to traverse among the nodes in both the directions. The pointer to the next node, each node of the linked list can also contain a pointer to previous node.

iii. Circular Linked List:

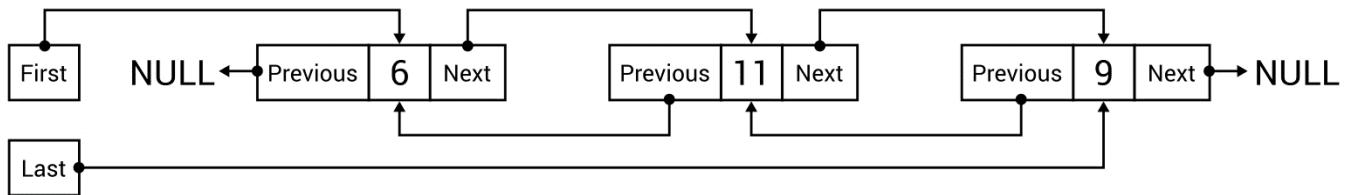
The pointer field in the last node is modified to point to the point the first node of the list.

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

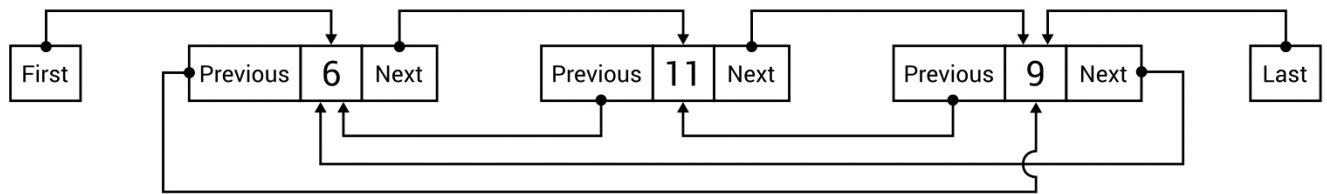
SINGLE-LINKED LIST



DOUBLE-LINKED LIST



CIRCULAR-LINKED LIST



A linked list is normally used to represent any data used in word-processing applications, also applied in different DBMS packages. For searching n^{th} node you have to traversed to $(n-1)$ preceding nodes.

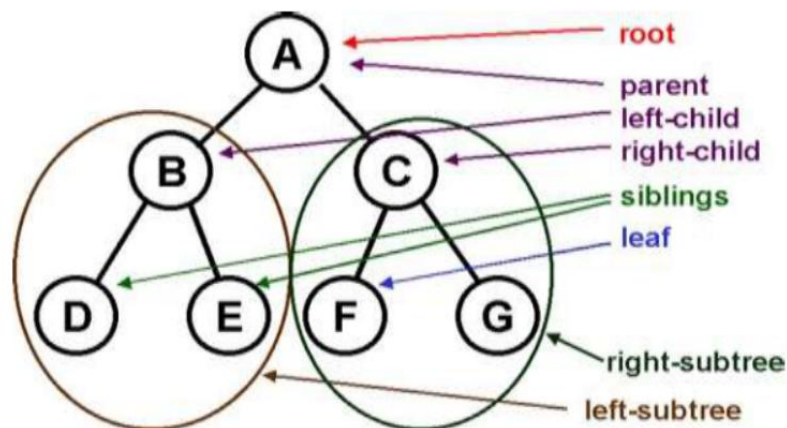
1.19. Non-linear data structure:

The non-linear data structure is one in which its elements do not form a sequence. i.e. elements do not have a unique successor and a unique predecessor.

The frequently used non-linear data structures are Tree & Graphs.

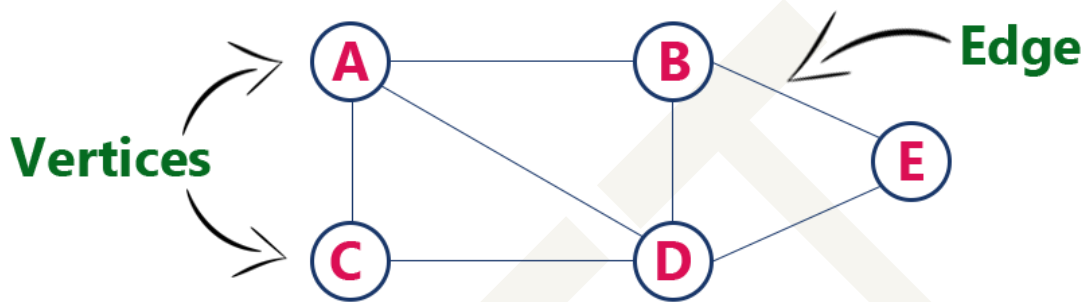
a. Trees

It maintains hierarchical relationship between various elements. The data structure which is used to represent the hierarchical relationship between the data elements with a special node at the top named ROOT, is called as TREE.



b. Graphs

It maintains random relationship or point-to-point relationship between various elements. The data structure which is used to represent the non-hierarchical relationship among pairs of the data elements is called as GRAPH. Graph consists of a pair of two non-empty sets – V&E. (Vertices and Edges)



1.20. Application of Data Structures

Stack

- Parsing of the Data
- Reversing the string or data
- Evaluation of the expressions
- Conversion of expressions i.e. – infix to postfix, infix to prefix postfix to infix and prefix to infix.
- Recursive function calling.
- Tower of Hanoi

Queue

- When a resource is shared among multiple consumers. Examples include CPU scheduling, Disk Scheduling.
- Sorting of the data
- Buffers on MP3 players and portable CD players, iPod playlist. Playlist for jukebox - add songs to the end, play from the front of the list.
- Computer systems must often provide a “holding area” for messages between two processes, two programs, or even two systems. This holding area is usually called a “buffer” and is often implemented as a queue.
- When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes. Examples include IO Buffers, pipes, file IO, etc.

LinkedList

- Remove element from middle of a collection, maintain order, no shifting. Add an element in the middle, no shifting.
- Linked lists can be used to implement Stacks, Queues, Graphs, etc.

DATA STRUCTURES (017013292)
Semester – II
Chapter Name: INTRODUCTION TO DATA STRUCTURE

Tree

- Used in many search applications where data is constantly entering/leaving, such as the map and set objects in many languages' libraries.
- Used in almost every high-bandwidth router for storing router-tables.
- Used in almost every 3D video game to determine what objects need to be rendered.
- Used in cryptographic applications to generate a tree of pseudo-random numbers.
- The library database in a library, a student database in a school or college, an employee database in a company, a patient database in a hospital or any database for that matter would be implemented using trees.
- The file system in your computer i.e. folders and all files, would be stored as a tree.
- Most probably, Google maps uses trees (apart from using graphs) for all the locations in it. Then it would be very easy to answer questions like a) which is the nearest restaurant to this place? List all schools which are present within 5 miles of radius from this place and so on.

Graph

- Networks (e.g., of computers or roads)
- Flow charts
- States of an automaton / program
- Tasks in some project (some of which should be completed before others), so edges correspond to prerequisites.
- For making electronics circuit
- Find out the shortest path in the networks